

# Manual de uso - Llama Desktop Launcher

Versão 0.2.6 · Português (Brasil)

## 1. O que o aplicativo faz

O Llama Desktop Launcher instala ou localiza o motor `llama.cpp`, encontra modelos locais em formato GGUF e abre a interface do `llama-server` no navegador. Você pode escolher acesso somente local, LAN ou Tailscale. Ao escolher a rede, a API exige uma chave. Os dados trocados com outros computadores trafegam pela rede escolhida.

O instalador do aplicativo **não inclui modelos**. O motor oficial também é baixado na primeira execução, ou você pode indicar uma instalação já existente. A versão do motor testada é `b11193`.

A interface tem três telas: **Llama**, para conversar e iniciar o servidor; **Modelos**, para encontrar GGUF e consultar recomendações; e **Configurações**, para preparar o motor, as pastas e os perfis. A navegação para Configurações permanece disponível em todas as telas.

## 2. Requisitos

| Plataforma / Distribuição             | Arquitetura     | Pacote oficial / Download direto              | Aceleração de GPU                                |
|---------------------------------------|-----------------|-----------------------------------------------|--------------------------------------------------|
| Windows (10 / 11)                     | x64             | `Llama Desktop Launcher Setup 0.2.6.exe`      | Vulkan, NVIDIA CUDA, AMD ROCm, Intel SYCL ou CPU |
| Linux (Debian, Ubuntu, Mint, Pop!_OS) | x64 (amd64)     | `llama-desktop-launcher_0.2.6_amd64.deb`      | Vulkan, CUDA ou CPU; ROCm/SYCL em x64            |
| Linux (Debian, Ubuntu)                | ARM64 (aarch64) | `llama-desktop-launcher_0.2.6_arm64.deb`      | Vulkan ou CPU                                    |
| Linux (Fedora, RHEL, openSUSE)        | x64 (x86_64)    | `llama-desktop-launcher-0.2.6.x86_64.rpm`     | Vulkan, CUDA ou CPU; ROCm/SYCL em x64            |
| Linux (Fedora, RHEL, openSUSE)        | ARM64 (aarch64) | `llama-desktop-launcher-0.2.6.aarch64.rpm`    | Vulkan ou CPU                                    |
| Linux (Portátil / Todas as distros)   | x64             | `Llama Desktop Launcher-0.2.6.AppImage`       | Vulkan, CUDA ou CPU; ROCm/SYCL em x64            |
| Linux (Portátil / Todas as distros)   | ARM64           | `Llama Desktop Launcher-0.2.6-arm64.AppImage` | Vulkan ou CPU                                    |
| macOS (Apple Silicon M1/M2/M3/M4)     | ARM64           | `Llama Desktop Launcher-0.2.6-arm64.dmg`      | Metal                                            |
| macOS (Intel)                         | x64             | `Llama Desktop Launcher-0.2.6.dmg`            | Metal                                            |

O espaço necessário depende do modelo escolhido. Reserve espaço para o arquivo GGUF, para o cache e para o motor. Modelos grandes podem exigir muito mais memória do que o tamanho do arquivo em disco. Se o computador não tiver GPU compatível, a execução pode cair para CPU e ficar lenta. No Linux, o pacote oficial do motor é compilado no Ubuntu; outras distribuições podem precisar de bibliotecas compatíveis. O GitHub Actions compila instaladores oficiais nativos para Windows x64, Linux x64/arm64 e macOS Intel/Apple Silicon.

### 3. Instalar o aplicativo

O repositório oficial do projeto disponibiliza o código-fonte e instaladores oficiais compilados e publicados automaticamente nas [Releases do GitHub](#), acompanhados de arquivos de verificação de integridade ([SHA256SUMS.txt](#)). Para instalar, obtenha o pacote correspondente à sua plataforma na página de Releases ou gere o instalador a partir do código-fonte seguindo o README.

#### Windows

1. Abra o instalador [Llama Desktop Launcher Setup 0.2.6.exe](#).
2. Escolha a pasta de instalação. O instalador trabalha por usuário, então não precisa colocar o aplicativo em [Program Files](#).
3. Marque ou desmarque a criação do atalho da área de trabalho.
4. Conclua a instalação e abra o aplicativo pelo Menu Iniciar ou pelo atalho.

#### Linux

- **DEB:** instale pelo gerenciador de pacotes da sua distribuição Debian/Ubuntu.
- **RPM:** instale pelo gerenciador de pacotes da sua distribuição Fedora/RHEL compatível.
- **Applmage:** torne o arquivo executável e abra-o. O Applmage é portátil e fica na pasta que você escolher; não é instalado pelo gerenciador de pacotes.

Os pacotes DEB/RPM usam os diretórios definidos pelo sistema e pelo gerenciador de pacotes. A escolha das pastas de modelos é feita dentro do aplicativo.

Quando o pacote da sua plataforma estiver disponível, use o DEB, RPM ou Applmage correspondente ao seu sistema. Arquivos portáteis experimentais de versões anteriores não substituem esses pacotes.

#### macOS

1. Abra o arquivo DMG.
2. Arraste o aplicativo para [Applications](#) ou para outra pasta onde você queira mantê-lo.
3. Abra o aplicativo pelo Finder.

As versões de desenvolvimento sem assinatura/notarização podem exigir autorização manual nas configurações de segurança do macOS. Não desative as proteções do sistema de forma geral.

### 4. Primeiro uso: motor llama.cpp

No primeiro início, a tela **Configurações** abre automaticamente quando o motor ainda não está pronto, ainda não há modelos, ou não existe um modelo selecionado e salvo. Depois de preparar o motor e escolher um modelo, as próximas aberturas vão para **Llama**. Você pode voltar às **Configurações** pela navegação a qualquer momento. A tela explica o que ainda falta; é possível concluir a configuração depois.

Na seção **Motor llama.cpp**, escolha uma das opções:

1. **Baixar motor oficial:** escolha o **Backend do pacote oficial** antes de clicar no botão. O aplicativo acessa a release [b11193](#) do projeto [ggml-org/llama.cpp](#) no GitHub e baixa o pacote da plataforma e do backend escolhidos. No Windows/Linux, as opções exibidas dependem da arquitetura e incluem Vulkan, NVIDIA CUDA, CPU e, em x64, AMD ROCm e Intel SYCL. No macOS, usa Metal. CUDA baixa também as bibliotecas oficiais correspondentes. O motor é guardado na pasta de dados do seu usuário, fora da instalação do aplicativo.

2. **Escolher pasta existente:** a pasta selecionada precisa conter `llama-server` (ou `llama-server.exe`). Use esta opção para aproveitar uma instalação já preparada, como a pasta `b11193-vulkan` do launcher Windows anterior.

O download requer internet. Escolha pelo tipo de GPU, independentemente da marca da CPU: Vulkan funciona com GPUs compatíveis de AMD, Intel ou NVIDIA; CUDA é para NVIDIA, ROCm para AMD e SYCL para Intel. Para uma RTX 4060 no Windows x64, comece por **NVIDIA CUDA 12.4** e mantenha o driver NVIDIA atualizado. ROCm e SYCL podem exigir runtimes e drivers adicionais. Se a release não tiver pacote para a arquitetura detectada, selecione um runtime compilado por você. O aplicativo verifica o SHA-256 quando a API da release fornece o digest. Se você já escolheu um motor e quiser substituí-lo, use **Escolher pasta existente** ou baixe novamente o pacote oficial na mesma tela.

Se você usa o launcher Windows anterior, selecione primeiro a pasta que contém seus GGUF. Em **Configurações > Perfis do launcher anterior**, clique em **Importar perfis TSV** e escolha `launcher-profiles.tsv` da pasta do launcher antigo. A importação associa perfis pelo caminho completo do modelo; se você moveu os GGUF, selecione a pasta antiga ou ajuste os perfis manualmente. Perfis já salvos no aplicativo não são sobrescritos. O resultado mostra quantos perfis foram importados, não localizados, já existentes ou inválidos.

## 5. Pastas e modelos GGUF

O aplicativo aceita **duas localizações** de modelos na tela **Configurações**. A primeira começa sugerida em `~/Models/GGUF`; clique em **Alterar** para escolher outro disco ou pasta. A segunda é opcional e pode ser adicionada ou removida. Você pode mudar as duas pastas depois. A tela **Modelos** procura arquivos `.gguf` dentro das subpastas. Nela, escolha o modelo que deseja usar; o aplicativo guarda a seleção e os ajustes associados ao caminho daquele arquivo.

Mesmo antes de baixar o primeiro modelo, **Modelos** mostra estimativas baseadas na RAM, CPU, GPU/VRAM detectáveis e espaço livre nas pastas de modelos. As faixas são pontos de partida conservadores, não uma garantia de compatibilidade ou desempenho. Confira sempre o tamanho do arquivo GGUF, a licença, os requisitos publicados pelo autor e o espaço disponível. Para MoE, considere o tamanho total dos pesos, não apenas os parâmetros ativos.

Para obter um modelo:

1. Em **Modelos**, explore as opções para abrir o Hugging Face no navegador: **Explorar todos os GGUF** para busca livre sem restrições; **Modelos Quantizados** (onde sugerimos Q4\_K\_M pelo equilíbrio de memória, mas você pode optar por Q5, Q8, etc.); **Modelos MoE** para arquiteturas esparsas com especialistas; ou **Modelos Densos (sem quantização)** para pesos originais F16/BF16. Você também pode acessar diretamente a busca de **todos os modelos GGUF**, **modelos quantizados**, **modelos MoE** e **modelos densos F16**.
2. Escolha um repositório e confira a **licença do modelo**, requisitos de RAM/VRAM, quantização e instruções do autor.
3. Baixe o arquivo `.gguf` para uma das pastas configuradas. Esta versão abre o Hugging Face no navegador; o download do arquivo é feito pelo navegador.
4. Volte ao aplicativo e atualize a lista de modelos. Em **Modelos**, confira a recomendação compatível com seu hardware e selecione o GGUF desejado.

Se um modelo usa projetor multimodal, mantenha um único arquivo `mmproj*.gguf` na mesma pasta. O aplicativo o associa automaticamente. Em modelos divididos em vários arquivos, todos os shards devem permanecer juntos; a lista mostra apenas o primeiro shard.

**Atenção:** GGUF descreve o formato, mas não garante que todo modelo seja compatível com esta versão do llama.cpp. Repositórios podem exigir uma versão mais nova do motor.

## 6. Tela Llama: iniciar uma conversa

1. Na lista **Modelo GGUF**, escolha um modelo já detectado nas pastas configuradas. Se ainda não existir um modelo selecionado e salvo, a tela inicial será **Configurações**; baixe ou localize um modelo primeiro na tela **Modelos**.

2. Para conversar dentro do aplicativo, clique em **Conversar aqui**, digite a mensagem e envie. Use Enter para enviar e Shift+Enter para inserir uma linha. O histórico não é salvo.
3. Clique em **Abrir no navegador** para iniciar **llama-server**, aguardar o carregamento e abrir a interface web. A tela mostra o endereço de acesso e as sessões configuradas.
4. Ao terminar, clique em **Parar servidor** no aplicativo.

Use **Abrir CLI no terminal** para iniciar **llama-cli** (**llama-cli.exe** no Windows) em um terminal separado, com o modelo e ajustes atuais. Converse e encerre pelo terminal. Pare o servidor ou a conversa integrada antes de iniciar a CLI externa.

O carregamento de um modelo grande pode levar alguns minutos. O registro do motor na parte inferior mostra mensagens e erros. Se a aba do navegador for fechada, o servidor continua ativo até você clicar em **Parar servidor** ou encerrar o aplicativo.

## 7. Parâmetros avançados

### Acesso, link e sessões simultâneas

Na seção **Acesso e sessões**, escolha **Somente este computador** ou um endereço LAN/Tailscale detectado. Defina de **1 a 8 sessões simultâneas**. O ajuste passa `--parallel` ao **llama-server**; mais sessões compartilham o contexto e a memória disponíveis, portanto comece com 1 ou 2 se o modelo for grande. Esse número é a capacidade simultânea do servidor, não a quantidade de tokens do contexto. O campo **Contexto (-c)** controla os tokens.

Ao iniciar em LAN/Tailscale, use **Mostrar chave**, **Copiar chave** ou **Nova chave**. Para trocar a chave, pare o servidor primeiro. O app abre a interface local por uma ponte autenticada. Use **Copiar link da API** para obter o endereço terminado em `/v1`, e **Copiar link da interface** para obter o endereço web na rede. Clientes remotos devem enviar **Authorization: Bearer SUA\_CHAVE**; o link por si só não concede acesso. Uma interface remota também precisa oferecer suporte a essa autenticação. Não coloque a chave no URL nem publique a chave em mensagens abertas.

O aplicativo não altera o firewall ao iniciar o servidor. No Windows, com o servidor LAN/Tailscale ativo, informe a **faixa de clientes permitidos** na seção **Firewall do Windows** e clique em **Criar regra para a porta ativa**. Confirme a solicitação de administrador do Windows. A regra limita o acesso ao executável **llama-server.exe**, à porta ativa e à faixa indicada. Confira a faixa antes de confirmar; se mudar de porta, crie outra regra. No Linux/macOS, siga as regras do firewall da distribuição. O servidor usa uma porta livre de 8080 a 8180. Prefira Tailscale ou uma LAN confiável, pois o HTTP direto na LAN não cifra o tráfego.

### Conversa integrada e CLI externa

Quem prefere o programa original pode clicar em **Abrir CLI no terminal**. O aplicativo abre **llama-cli** (**llama-cli.exe** no Windows) em uma janela de terminal separada, com o modelo e os ajustes atuais. Use o próprio terminal para conversar e encerrá-lo; o botão **Encerrar conversa** controla apenas a conversa interna. Pare o servidor ou a conversa interna antes de abrir a CLI externa. Um terminal gráfico precisa estar instalado no Linux. O terminal externo pode continuar aberto depois que o launcher for fechado.

Os parâmetros avançados aparecem na tela **Llama** dentro do acordeão **Ajustes avançados deste modelo**, fechado inicialmente. Abra-o para revisar os valores, aplicar uma sugestão inicial ou comparar microbatch. Clique em **Salvar ajustes deste modelo** para guardar o perfil do GGUF selecionado. A tela **Configurações** também oferece o seletor de modelo/perfil e mantém seus ajustes abertos para edição. Os parâmetros são salvos por caminho de modelo.

**Aplicar sugestão inicial** preenche os campos com base na RAM, no número de threads de CPU e nos dados do GGUF. A estimativa pode incluir GPU/VRAM se o sistema conseguir detectá-las. É um ponto de partida, não uma garantia de velocidade ou de uso exato de memória. Revise os valores e salve o perfil quando quiser mantê-los.

| Campo              | Argumento do llama.cpp   | Como usar                                                                                                                               |
|--------------------|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Camadas na GPU     | <code>-ngl</code>        | <code>auto</code> deixa o motor ajustar; <code>all</code> tenta usar GPU para todas as camadas; <code>0</code> usa CPU.                 |
| Dispositivo        | <code>-dev</code>        | Vazio deixa a escolha automática. Use um identificador como <code>Vulkan0</code> quando houver mais de uma GPU.                         |
| Contexto           | <code>-c</code>          | Número de tokens mantidos na conversa. Valores maiores consomem mais memória.                                                           |
| Camadas MoE na CPU | <code>-ncmoe</code>      | Mantém os pesos de especialistas das primeiras camadas na RAM. Só faz sentido para modelos MoE; vazio desativa esse ajuste.             |
| Cache K / V        | <code>-ctk / -ctv</code> | <code>f16</code> prioriza fidelidade; <code>q8_0</code> reduz memória com pequeno custo. Outros formatos devem ser testados por modelo. |
| Flash Attention    | <code>-fa</code>         | <code>auto</code> , <code>on</code> ou <code>off</code> ; disponibilidade depende do motor/backend.                                     |
| Threads CPU        | <code>-t</code>          | Número de threads para partes executadas na CPU. Vazio usa o padrão do motor.                                                           |
| Batch              | <code>-b</code>          | Tamanho lógico de processamento do prompt.                                                                                              |
| Microbatch         | <code>-ub</code>         | Tamanho físico de cada bloco. Não pode ser maior que batch.                                                                             |

O motor recebe também `-fit on` para adaptar a carga à memória disponível. Um modelo denso não deve receber `-ncmoe`; deixe o campo vazio. Para um MoE que não cabe totalmente na VRAM, teste o número de camadas MoE na CPU e observe o desempenho e o consumo de RAM.

## 8. Analisar recomendações e testar o desempenho

Na tela **Modelos**, o aplicativo prepara um prompt com as informações de hardware disponíveis, o modelo selecionado e seus ajustes. Use **Copiar prompt de análise** para colá-lo em outro lugar, ou **Analisar com modelo selecionado** para enviá-lo pela conversa integrada ao modelo atual. Se ainda não houver modelo, você pode copiar o prompt e usar as sugestões locais; a análise com IA fica disponível depois de selecionar um GGUF.

A resposta do modelo é uma recomendação para você revisar. Ela não altera nem salva os parâmetros automaticamente. Confira compatibilidade, memória, licença e os resultados reais antes de aplicar qualquer mudança. As sugestões locais também são estimativas; o benchmark abaixo mede o desempenho no computador.

Para medir o microbatch depois de selecionar e carregar um GGUF:

1. Na tela **Llama**, abra **Ajustes avançados deste modelo**. Se desejar uma sugestão para vários campos, clique em **Aplicar sugestão inicial**; isso não executa o modelo.
2. Pare o servidor ou conversa ativa e clique em **Comparar microbatch**. O `llama-bench` testa o valor atual e outro de até 128, limitado pelo batch. Quando os dois valores coincidem, há apenas um teste.

3. Aguarde os resultados de processamento de prompt e geração, em tokens por segundo. O aplicativo coloca no campo **Microbatch** o valor com melhor pontuação; os outros campos permanecem como estavam.

4. Revise os campos e clique em **Salvar ajustes deste modelo** para manter o resultado sem iniciar o modelo.

O teste pode carregar o modelo mais de uma vez e levar vários minutos. Ele não salva o perfil sozinho nem otimiza contexto, cache, camadas de GPU ou camadas MoE. Esses valores continuam ajustáveis manualmente.

## 9. Solução de problemas

| Sintoma                     | O que verificar                                                                                                                        |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Não há modelos na lista     | Confirme as duas pastas e a extensão <a href="#">.gguf</a> ; clique em <b>Atualizar lista</b> .                                        |
| O download do motor falhou  | Confira internet, espaço livre e acesso ao GitHub. Tente de novo ou use <b>Escolher pasta existente</b> .                              |
| “Pacote sem llama-server”   | O arquivo da release pode ter mudado; escolha um runtime compatível manualmente.                                                       |
| O servidor encerra ao abrir | Consulte <b>Registro do motor</b> ; reduza contexto/batch, experimente <a href="#">-ngl auto</a> ou menos camadas de GPU.              |
| GPU não é usada             | Confira o backend escolhido, a compatibilidade da GPU e seus drivers; no macOS, confirme o pacote Metal. Verifique mensagens do motor. |
| Benchmark falha             | Confirme que a pasta do runtime inclui <a href="#">llama-bench</a> ; tente parâmetros mais conservadores.                              |
| Modelo não carrega          | Verifique todos os shards, compatibilidade com <a href="#">b11193</a> , licença/instruções do repositório e memória disponível.        |

## 10. Dados, atualização e desinstalação segura

As preferências e perfis ficam em [settings.json](#) na pasta de dados do usuário do aplicativo. O motor baixado fica sob [engines/](#) nessa mesma área. Os modelos permanecem nas pastas que você escolheu. O aplicativo não aceita uma pasta de modelos dentro da instalação nem dentro de sua pasta de dados internos.

Antes de desinstalar, volte ao aplicativo, clique em **Parar servidor** se estiver ativo e feche o aplicativo normalmente. No Windows, o desinstalador verifica se ele ainda está aberto e pede que você o feche; **Tentar novamente** prossegue após o fechamento. Ele não força o encerramento do app. Use **Configurações > Aplicativos instalados**. O desinstalador remove o aplicativo e seus atalhos, mas **preserva por padrão os modelos, perfis e o motor baixado**. Não use opções avançadas de remoção de dados sem verificar o conteúdo da pasta de dados.

No Linux, use o gerenciador de pacotes para remover DEB/RPM, ou apague o arquivo AppImage. No macOS, mova o aplicativo para o Lixo. Em ambos, os dados de usuário e os GGUF permanecem para revisão manual. Se você quiser apagá-los, faça isso separadamente depois de confirmar o caminho e fazer cópia de [settings.json](#) se desejar preservar os perfis. Nunca apague uma pasta de modelos apenas porque ela foi selecionada no aplicativo.

## 11. Créditos

O llama.cpp foi iniciado por Georgi Gerganov e é mantido pelos colaboradores da ggml-org, sob licença MIT. Este launcher independente foi desenvolvido para Célio com Codex, com base no launcher WinForms anterior, e está sob **licença MIT**, com copyright de Célio. A publicação pública do repositório está planejada; os instaladores exigem publicação separada. Os autores do llama.cpp não participam nem endossam este aplicativo. Cada modelo tem licença própria.